

American Sign Language Detection

1. Project Overview

In today's world, effective communication is key to fostering inclusivity, especially for people with disabilities. One such group is the Deaf and Hard of Hearing (DHH) community, who rely on sign language to communicate. Despite advancements, there are still challenges in bridging communication gaps between people who use sign language and those who don't understand it. The objective of this project is to develop a **Real-time Sign Language Recognition System** that can automatically identify American Sign Language (ASL) hand gestures and convert them into text or. This system will be implemented using computer vision and deep learning techniques.

2. Real-World Problem

In public spaces, classrooms, hospitals, and even customer service centers, Deaf and Hard of Hearing individuals often face difficulties in communicating with people who are not familiar with sign language. This can lead to feelings of isolation or frustration. Sign language interpreters are not always available, and face-to-face communication can be slow and inefficient. A real-time system that can recognize ASL gestures and convert them into a readable or audible form can bridge this communication gap, enabling better interaction, and improving accessibility for individuals with hearing impairments.

This project aims to solve these challenges by building a deep learning-based model capable of recognizing individual ASL letters, which can then be used in various applications like virtual assistants, accessibility tools, and communication aids.

3. Data Collection and Preparation

- **Dataset Overview:** The dataset used in this project consists of images depicting ASL hand gestures representing the English alphabet. The images are organized into different directories for each letter of the alphabet, each containing multiple hand gesture images. The dataset is divided into training and testing subsets:
 - **Training Dataset:** Used to train the deep learning model.
 - **Testing Dataset:** Used to evaluate the performance of the trained model.

```
def load_images_from_directory(data_dir, img_size=64):
    x, y = [], []
    labels = sorted(os.listdir(data_dir)) # Sorted to ensure consistent mapping
    label_map = {label: idx for idx, label in enumerate(labels)}

    print(f"Loading images from: {data_dir}")
    for label in labels:
        path = os.path.join(data_dir, label)
        if not os.path.isdir(path):
            print(f"Skipping non-directory file: {path}")
            continue
        print(f"Processing class '{label}'...")
        for img_file in os.listdir(path):
            img_path = os.path.join(path, img_file)
            if not img_file.lower().endswith(('.png', '.jpg', '.jpeg')): # Skip invalid formats
                print(f"Skipping non-image file: {img_path}")
                continue
            img = cv2.imread(img_path, cv2.IMREAD_COLOR)
            if img is None:
                print(f"Failed to read image: {img_path}")
                continue
            img = cv2.resize(img, (img_size, img_size)) / 255.0 # Normalize pixel values
            x.append(img)
            y.append(label_map[label])

    x = np.array(x, dtype="float32")
    y = to_categorical(np.array(y), num_classes=len(labels)) # One-hot encoding
    print(f"Loaded {len(x)} images from {data_dir}")
    return x, y, label_map
```

- **Image Preprocessing:**

- The images are resized to a consistent size of **64x64 pixels** to ensure uniformity.
- Pixel values are normalized to a range between 0 and 1.
- Data augmentation is applied to introduce variability in the training data, such as random rotations, shifts, zooms, and horizontal flips. This helps improve the robustness of the model and reduces overfitting.

```
# Enhanced Data Augmentation
datagen_train = ImageDataGenerator(
    rotation_range=30, # Increase rotation range
    width_shift_range=0.3, # Increase width shift
    height_shift_range=0.3, # Increase height shift
    shear_range=0.2, # Add shear
    zoom_range=0.3, # Increase zoom
    horizontal_flip=True,
    fill_mode='nearest'
)
```

- **Class Label Encoding:** Each hand gesture is mapped to a unique label corresponding to a letter in the alphabet. These labels are then one-hot encoded for use in classification.

4. Model Architecture

The core of this project is a **Convolutional Neural Network (CNN)** built using the **MobileNetV2** architecture, which is efficient and lightweight, suitable for real-time applications where processing speed is critical.

```
# Define the model architecture (with MobileNetV2 as base)
base_model = MobileNetV2(weights='imagenet', include_top=False, input_shape=(img_size, img_size, 3))
for layer in base_model.layers:
    layer.trainable = False # Freeze pretrained layers

x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.5)(x)
output = Dense(len(label_map), activation='softmax')(x)

model = Model(inputs=base_model.input, outputs=output)
model.compile(optimizer=Adam(learning_rate=0.0001), loss='categorical_crossentropy', metrics=['accuracy'])
model.summary()
```

- **MobileNetV2** is chosen as the base model because it's a pre-trained model known for its efficient performance in real-time applications, particularly for image classification tasks. The pre-trained model is initially frozen to retain its learned features, which are later fine-tuned to adapt to the specific ASL dataset.
- **Model Layers:**
 - **Base Model:** The pre-trained MobileNetV2 acts as the feature extractor.
 - **Global Average Pooling Layer:** Reduces the dimensionality of the feature maps to produce a fixed-size output.
 - **Dense Layers:** These layers perform classification, with dropout layers added to prevent overfitting.
 - **Softmax Output Layer:** The final layer uses softmax activation to output probabilities for each of the 26 ASL letters, allowing the system to predict the most likely gesture.

5. Model Training and Fine-tuning

- **Data Augmentation:** A set of transformations is applied to the training data, including rotation, width/height shifts, and zooming. This increases the diversity of the dataset, helping the model generalize better and avoid overfitting.
- **Fine-tuning:** Initially, the pre-trained MobileNetV2 layers are frozen to preserve the learned features from ImageNet. After the model is trained on the ASL dataset, the last 20 layers of MobileNetV2 are unfrozen for fine-tuning, allowing the model to learn more specific features of ASL hand gestures.
- **Class Weighting:** The system computes class weights to account for any imbalance in the dataset, ensuring that the model gives equal attention to less-represented classes.
- **Callbacks:**
 - **ModelCheckpoint:** Saves the model with the best validation loss.
 - **ReduceLROnPlateau:** Reduces the learning rate when the validation loss plateaus, allowing the model to refine its learning.
 - **EarlyStopping:** Stops the training process if the validation accuracy doesn't improve, preventing overtraining.

```
# Fine-tune MobileNetV2
base_model.trainable = True # Unfreeze all layers of the base model

# Fine-tune from a certain layer to retain lower-level features
fine_tune_at = 100 # Unfreeze all layers from this point onward
for layer in base_model.layers[:fine_tune_at]:
    layer.trainable = False

model.compile(optimizer=Adam(learning_rate=0.0001), loss='categorical_crossentropy', metrics=['accuracy'])
```

6. Model Evaluation

The model's performance is evaluated using the testing dataset. The system calculates the accuracy of predicting each ASL sign. Additionally, a **classification report** is generated, which includes precision, recall, and F1 score for each class (ASL letter), providing a detailed view of the model's performance.

- **Confusion Matrix:** A confusion matrix is generated to visualize misclassifications, helping identify where the model is making errors.
- **Test Image Prediction:** The trained model is used to predict hand gestures from new test images. The predicted label is displayed alongside the confidence probability, ensuring users can trust the model's output.

```
# Evaluate the model
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
accuracy = math.floor(test_acc * 100)

print(f"Test Accuracy: {accuracy} %")
```

```
558/558 - 10s - 18ms/step - accuracy: 0.9998 - loss: 0.0011
Test Accuracy: 99 %
```

7. Deployment and Real-Time Use Case

Once trained, the model can be deployed on various platforms like mobile applications, web apps, or desktop applications to recognize ASL gestures in real-time. The system can be used in the following real-world scenarios:

- **Real-time Communication Aids:** For use in public spaces, schools, or hospitals, the system can translate ASL signs into text or speech in real time, allowing better communication between Deaf individuals and the general public.
- **Virtual Assistants:** Integrating the model into voice assistants or chatbots can help enhance accessibility, enabling users to input commands through sign language instead of voice or text.
- **Accessibility Tools:** The system can be part of an accessibility toolkit for various devices, enabling Deaf individuals to interact more easily with technology.

8. Results and Visualization

- **Model Accuracy and Loss Curves:** During training, the model's accuracy and loss are plotted to assess its learning progress. The curves show the improvements in accuracy and reductions in loss during both training and validation phases.
- **Prediction Results:** After training, the system predicts ASL gestures from new images. These predictions, along with the probabilities, are displayed, allowing the user to confirm the accuracy of the system.

9. Conclusion and Future Scope

This Sign Language Recognition System offers a solution to breaking communication barriers between the Deaf community and those who do not know sign language. With a high degree of accuracy and the ability to operate in real-time, the system has the potential to be widely implemented in applications ranging from healthcare to customer service.

In the future, the model can be extended to recognize complete sign language phrases rather than just individual letters. Furthermore, the system can be adapted to recognize other types of sign languages, such as British Sign Language (BSL), making it a universal tool for communication.